

Computação Orientada a Objetos

Aula 09 – Parte 2

Prof. Bernardo Teixeira de Miranda

E-Mail: bernardo.t.miranda@ufv.br

Sala: LAE 123



Diferença entre *extends* e *implements*

- Em Java, *extends* e *implements* são usados para reaproveitar ou padronizar comportamentos entre classes.
- A diferença principal entre as duas é:
 - *extends* é usado para herdar de uma classe.
 - *implements* é usado para **seguir o contrato** de uma interface.
- De forma bem simples:

Palavra	Ideia	Exemplo
<i>extends</i>	“É um tipo de”	Cachorro extends Animal
<i>implements</i>	“Sabe fazer” / “segue uma regra”	Email implements Notificavel



O que é extends?

- Usa-se extends quando você deseja aplicar herança á sua classe.
 - Quando falamos que a **classe A** estende a **classe B**, significa que A herda alguns (ou todos) métodos e atributos da classe B.
 - Os únicos métodos e atributos que não são herdados são os que possuem o modificador de acesso *private*. Pode-se aplicar diversos níveis de herança. Por exemplo:
 - `class A extends B { };`
 - `class B extends C { };`
 - `class C extends D { };`
 - Ao fazer isso, os membros com modificador de acesso *public* das classes D, C e B são acessíveis na classe A. Os membros com modificador de acesso *protected* da classe B também são acessíveis na classe A.



O que é *extends*?

- Em Java é possível herdar apenas de uma classe. Não existe herança múltipla.
- Sabemos que ***extends*** é utilizado quando uma classe herda características de outra classe.
- Aqui, Cachorro herda o método comer() da classe Animal.

```
class Animal {  
    public void comer() {  
        System.out.println("O animal está comendo.");  
    }  
}  
  
class Cachorro extends Animal {  
    public void latir() {  
        System.out.println("O cachorro está latindo.");  
    }  
}
```



O que é *implements*?

- Usa-se implements quando você deseja implementar uma interface.
- Não implementa-se classes. Implementa-se apenas interfaces.
- Uma interface “firma um contrato” entre classes em que define comportamentos (métodos) que devem ser sobrescritos pela classe que os herda (se essa for uma classe concreta).
- Uma interface pode conter métodos e constantes. Constantes em Java são definidas pelas palavras *static* e *final*.
 - Ex: `public static final String NOME = “xpto”;`
- Os métodos de uma interface não tem corpo. Eles tem apenas a sua definição.



```
public interface Imprimivel {  
    public void imprime();  
}
```



O que é *implements*?

- Pode-se também aplicar herança entre interfaces (exclusivamente):

```
public interface Monstro {  
    public void ameacar();  
}  
  
public interface MonstroPerigoso extends Monstro {  
    public void destruir();  
}  
  
public class Godzilla implements MonstroPerigoso {  
    @Override  
    public void ameacar() {  
        //implementação  
    }  
  
    @Override  
    public void destruir() {  
        //implementação  
    }  
}
```



O que é *implements*?

- Então usamos *implements* quando uma classe precisa seguir uma interface.
- A interface diz quais métodos a classe precisa ter.

```
interface Notificavel {  
    void enviarMensagem(String mensagem);  
}
```

- Essa interface está dizendo:
 - Toda classe que for *Notificavel* **precisa** ter o método *enviarMensagem*.



O que é *implements*?

```
class Email implements Notificavel {  
    @Override  
    public void enviarMensagem(String mensagem) {  
        System.out.println("Enviando e-mail: " + mensagem);  
    }  
}  
  
class SMS implements Notificavel {  
    @Override  
    public void enviarMensagem(String mensagem) {  
        System.out.println("Enviando SMS: " + mensagem);  
    }  
}
```



Para que serve o *implements*?

- O benefício aparece quando queremos que classes diferentes sejam usadas do mesmo jeito.
 - Por exemplo, Email e SMS são classes diferentes. Mas as duas fazem a mesma ação: enviar uma mensagem. Por isso, as duas podem implementar a interface Notificavel.

```
interface Notificavel {  
    void enviarMensagem(String mensagem);  
}
```



Para que serve o *implements*?

- Agora, qualquer classe que implementar **Notificavel** será obrigada a ter o método *enviarMensagem*.

```
class Email implements Notificavel {
    @Override
    public void enviarMensagem(String mensagem) {
        System.out.println("Enviando e-mail: " + mensagem);
    }
}

class SMS implements Notificavel {
    @Override
    public void enviarMensagem(String mensagem) {
        System.out.println("Enviando SMS: " + mensagem);
    }
}
```



Para que serve o *implements*?

- Agora podemos criar uma classe que trabalha com qualquer tipo de notificação:

```
class ServicoNotificacao {  
    public void enviar(Notificavel notificacao, String mensagem) {  
        notificacao.enviarMensagem(mensagem);  
    }  
}
```

- Perceba que o método recebe um `Notificavel`, e não um `Email` ou um `SMS`. Isso significa que ele aceita qualquer classe que implemente essa interface.

```
public class Main {  
    public static void main(String[] args) {  
        ServicoNotificacao servico = new ServicoNotificacao();  
  
        servico.enviar(new Email(), "Sua compra foi aprovada.");  
        servico.enviar(new SMS(), "Seu código é 1234.");  
    }  
}
```



Para que serve o *implements*?

- Logo então, a classe ***ServicoNotificacao*** não precisa saber qual é o tipo real do objeto. Ela não precisa perguntar:
 - É um e-mail?
 - É um SMS?
 - É um WhatsApp?
- Ela só precisa saber:
 - Esse objeto é ***Notificavel***?
 - Então ele tem o método ***enviarMensagem***.
 - Ele permite criar um código mais genérico, que funciona com várias classes diferentes.



Duvidas?

